

Project Documentation: Dual-Edge Mealy Detector Unit

Digital Design Lab

Project Overview

This lab implements a **Mealy Finite State Machine (FSM)** for dual-edge (any-edge) detection in Verilog HDL[cite: 31]. The circuit monitors an input signal (`level`) and combinationaly asserts a pulse (`tick`) immediately upon detecting either a rising edge ($0 \rightarrow 1$) or a falling edge ($1 \rightarrow 0$)[cite: 31]. Because it is designed as a Mealy FSM, the output `tick` reacts instantly to input transitions prior to the next active clock edge[cite: 31].

Specifications & Functionality

- **States:**
 - **ZERO** (`1'b0`): Low input tracking state[cite: 31]. Output `tick` = 1 combinationaly when `level` = 1[cite: 31]. Transitions to **ONE** if `level` = 1[cite: 31].
 - **ONE** (`1'b1`): High input tracking state[cite: 31]. Output `tick` = 1 combinationaly when `level` = 0[cite: 31]. Transitions back to **ZERO** if `level` = 0[cite: 31].
- **Asynchronous Reset:** Active-high reset forces current state register to **ZERO** immediately on `posedge reset`[cite: 31].
- **Dual-Edge Mealy Output:** Asserts high (`tick` = 1) if (`current_state` == **ZERO** $\wedge$ `level` == `1'b1`) OR (`current_state` == **ONE** $\wedge$ `level` == `1'b0`)[cite: 31].

Port Summary

Port Name	Direction	Type	Width	Description
<code>clk</code>	Input	wire	1	Clock input signal[cite: 31, 32]
<code>reset</code>	Input	wire	1	Asynchronous active-high reset[cite: 31, 32]
<code>level</code>	Input	wire	1	Input level signal to monitor[cite: 31, 32]
<code>tick</code>	Output	reg	1	Combinational pulse on any edge[cite: 31, 32]

Table 1: Dual-Edge Mealy Detector Port Interfaces

Verilog Source Code

Design Module (`edge_mealy.v`)

```
module edge_mealy (  
    input wire clk,  
    input wire reset,  
    input wire level,
```

```

    output reg    tick

);

parameter ZERO = 1'b0;
parameter ONE  = 1'b1;

reg current_state;
reg next_state;

// State Register with Asynchronous Reset
always @(posedge clk or posedge reset) begin
    if (reset)
        current_state <= ZERO;
    else
        current_state <= next_state;
end

// Next State Logic
always @(*) begin
    case (current_state)
        ZERO: begin
            if (level)
                next_state = ONE;
            else
                next_state = ZERO;
        end

        ONE: begin
            if (level)
                next_state = ONE;
            else
                next_state = ZERO;
        end

        default:
            next_state = ZERO;
    endcase
end

// Dual-Edge Mealy Output Logic
always @(*) begin
    if ((current_state == ZERO && level == 1'b1) ||
        (current_state == ONE  && level == 1'b0))
        tick = 1'b1;
    else
        tick = 1'b0;
end

endmodule

```

Testbench Module (tb_edge_mealy.v)

```

`timescale 1ns / 1ps

module tb_edge_mealy;

```

```
reg clk;
reg reset;
reg level;

wire tick;

edge_mealy DUT (
    .clk    (clk),
    .reset  (reset),
    .level  (level),
    .tick   (tick)
);

// Clock Generation (10ns Period)
initial begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end

// Test Stimulus
initial begin
    reset = 1'b1;
    level = 1'b0;
    #10;

    reset = 1'b0;
    #10;

    level = 1'b1;
    #40;

    level = 1'b0;
    #40;

    level = 1'b1;
    #20;

    level = 1'b0;
    #20;

    $finish;
end

// Output Display Monitor
initial begin
    $monitor(
        "Time = %0t | clk = %b | reset = %b | level = %b | tick = %b",
        $time,
        clk,
        reset,
        level,
        tick
    );
end

endmodule
```

Simulation & Verification

Execution Command

To execute batch simulation using Cadence Xcelium or ModelSim:

```
xrun edge_mealy.v tb_edge_mealy.v
```

Expected Simulation Output

The simulation log confirms that **tick** asserts immediately when **level** transitions, whether rising ($0 \rightarrow 1$) or falling ($1 \rightarrow 0$)[cite: 31, 32]:

```
Time = 0 | clk = 0 | reset = 1 | level = 0 | tick = 0
Time = 5 | clk = 1 | reset = 1 | level = 0 | tick = 0
Time = 10 | clk = 0 | reset = 0 | level = 0 | tick = 0
Time = 20 | clk = 0 | reset = 0 | level = 1 | tick = 1
Time = 25 | clk = 1 | reset = 0 | level = 1 | tick = 0
Time = 60 | clk = 0 | reset = 0 | level = 0 | tick = 1
Time = 65 | clk = 1 | reset = 0 | level = 0 | tick = 0
Time = 100 | clk = 0 | reset = 0 | level = 1 | tick = 1
Time = 105 | clk = 1 | reset = 0 | level = 1 | tick = 0
Time = 120 | clk = 0 | reset = 0 | level = 0 | tick = 1
Time = 125 | clk = 1 | reset = 0 | level = 0 | tick = 0
```